

Fridge Recipe Tracker — Engineering Report

Team Project Part 1 · Programming Fundamentals · AI-assisted, food-waste-aware recipe planner (procedural Python CLI)

Lab group: [LabGroup] Team: [TeamNumber]

Submission: 14 October 2026

Members: [Name (Student ID), Name (Student ID), ...]

Repository: github.com/clouden56/PROJECTKITCHEN

1. Problem and Objectives

Households routinely discard food that was simply forgotten until it expired, then spend more on groceries or takeaway, and face daily "what can I cook?" decision fatigue. **Fridge Recipe Tracker** is a command-line kitchen assistant that records fridge and pantry items with expiry dates and asks Google Gemini for a recipe that uses the most urgent items first. The AI output is never trusted blindly: a deterministic rule engine accepts, flags or rejects every suggestion before it reaches the user.

- **Track** inventory (name, quantity, unit, location, expiry) in flat JSON files that survive restarts and container rebuilds.
- **Generate** recipes with AI under the user's meal type, time limit, allergies and must-include ingredients, using a strict JSON response schema.
- **Decide** with business rules: allergen gate (with a synonym map, e.g. *dairy* → *milk, cheese, butter*), ≥ 50 % ingredient-match threshold, cook-time limit, must-include check, and a minimum of two inventory items before calling the API.
- **Measure impact**: an expiry-priority score (expired/today +40, 1–3 days +30, 4–7 days +15, otherwise +5, capped at 100) and an estimate of grams of food diverted from waste.

2. Architecture

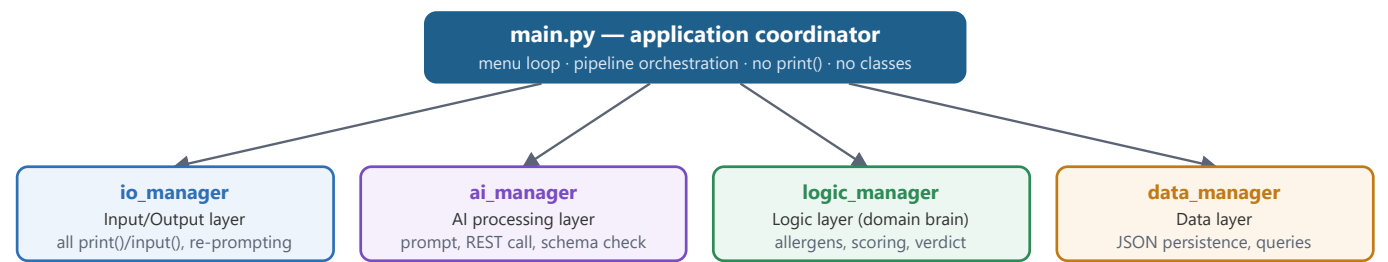


Figure 1. Hub-and-spoke modular architecture. No manager imports another manager; only `main.py` composes them, so each layer can be tested and replaced in isolation.

Module	Single responsibility	Key procedural functions (all typed; bold = framework API)
io_manager	Collect and validate every keystroke and return typed dicts; render every line of output. The only module that calls <code>print()</code> or <code>input()</code> .	<code>prompt_recipe_generation_criteria</code> , <code>prompt_new_ingredient</code> , <code>prompt_history_filter</code> , <code>display_inventory</code> , <code>display_recipe_card</code> , <code>display_recipe_history</code>
ai_manager	Build the prompt, call the Gemini REST API with a model cascade, and validate the JSON schema. Contains no domain rules.	<code>build_prompt</code> , <code>call_api</code> , <code>parse_response</code> , <code>validate_response</code> , <code>process</code> , <code>classify_api_error</code>
logic_manager	Apply the business rules to validated AI output and produce a verdict: <code>ACCEPTED</code> , <code>FLAGGED</code> or <code>REJECTED</code> .	<code>evaluate</code> , <code>score</code> , <code>route</code> , <code>evaluate_allergen_safety</code> , <code>calculate_ingredient_match_ratio</code> , <code>check_mandatory_ingredients</code>
data_manager	Load, save and query flat JSON files; survive missing, corrupt or unwritable files.	<code>save</code> , <code>load</code> , <code>query(filter_fn)</code> , <code>load_inventory</code> , <code>save_inventory</code> , <code>quarantine_corrupt_file</code> , <code>query_items_by_expiry</code>

Design conventions. The program is 100 % procedural: there are zero `class` definitions, and an automated test scans every `.py` file to enforce this. Layers exchange only plain dict/list values. Failures are returned as values (`(success, result, error_message)` tuples, defaults or `False`), so an exception never crosses a layer boundary and the menu loop cannot crash. Diagnostics go to `data/app.log` through logging, keeping the terminal UI clean.

3. Data Flow Diagram and Exception Handling Matrix

The unit of work (the **record**) is a recipe request. Every request passes through `ai_manager`, and the validated AI output drives every decision in `logic_manager`; inventory items are reference data that the request carries. Figure 2 follows one request (menu option 4): each numbered box is a **state** of the data, each arrow the function that transforms it, and swimlanes show which layer owns each state.

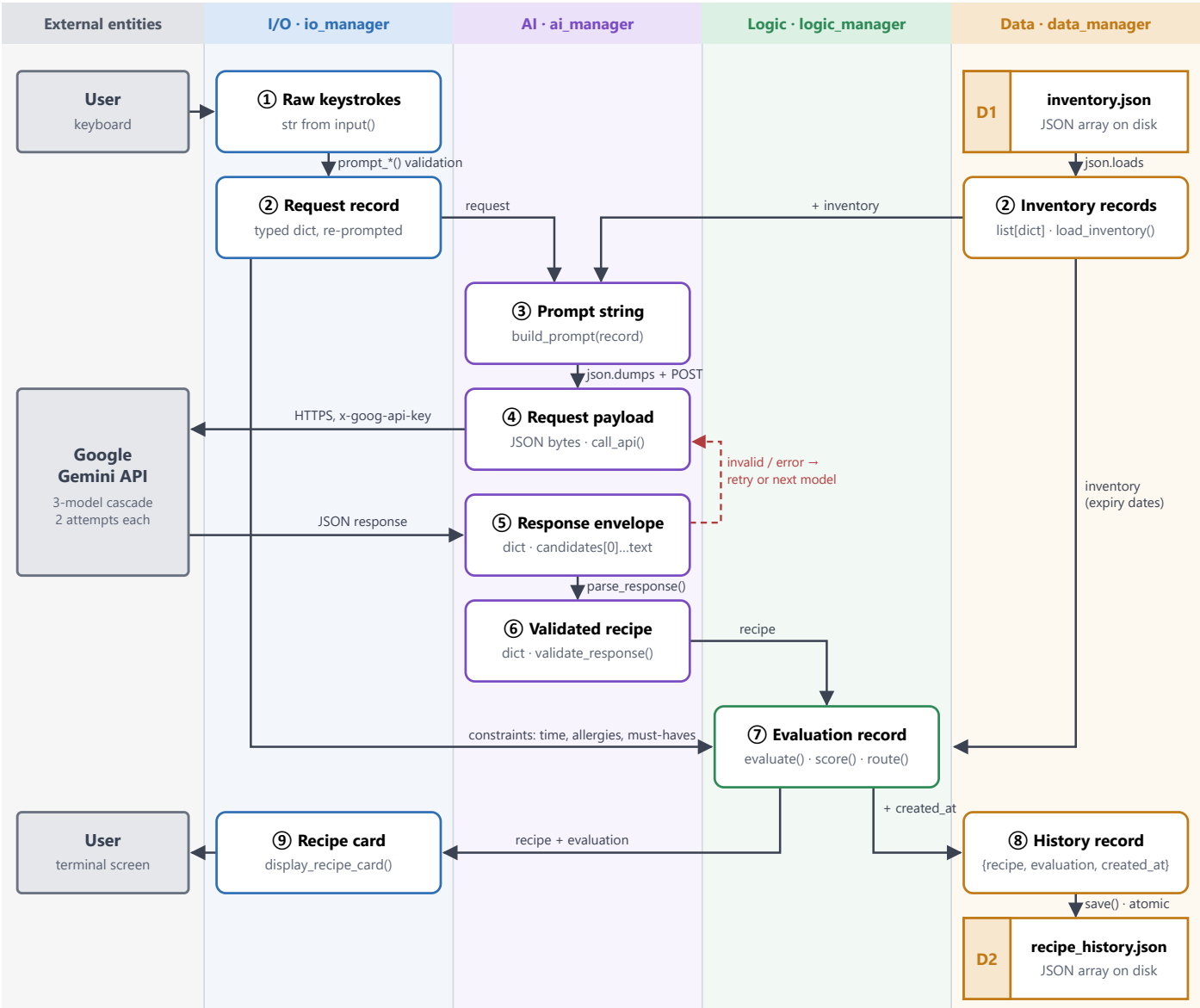


Figure 2. Data flow from user input → AI payload → file storage. The dashed red loop is the recovery path for a failed or invalid response; if every attempt fails, the user sees an error, **nothing is written to D2**, and control returns to the menu.

Failure class	Detected by	Handling & recovery → what the user sees	Detail · tests
API connection failure	call_api(): URLError, TimeoutError, HTTPError; classify_api_error()	Logged and returned as (False, None, msg), never raised. Transient errors back off and retry, then fall back to the next model; a bad key aborts at once; nothing is saved. → [ERROR] "AI API service temporarily unavailable..."	#1–5 · T9 T12
Malformed API response	parse_response(), validate_response()	Prose, truncated, empty or blocked replies and schema violations (missing keys, wrong types, out-of-range values) are rejected with a reason and retried. Invalid data never reaches logic_manager.	#6–10 · T4 T8 T9
Corrupt or missing data file	load_json_data(), save_json_data()	A missing file returns []. A corrupt file is logged, moved to a .bak backup, and [] is returned. Writes are atomic; write failures return False. → The app keeps running.	#11–16 · T3 T10
Invalid user input	io_manager prompt_*() loops	Type, range and required-field checks re-prompt until the value is valid; only typed dicts leave io_manager. Ctrl+C or EOF exits cleanly with code 0.	#17–21 · T11 T12

Table 3. Exception handling matrix (summary); every scenario is broken down in Section 4 with the automated test that proves it.

Inventory path (menu option 2): user input → file storage

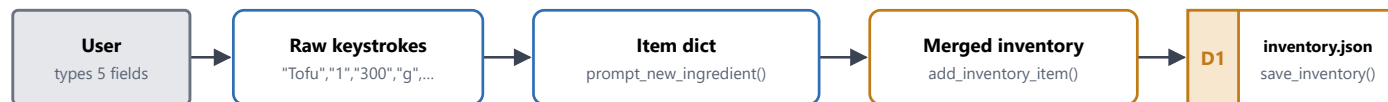


Figure 3. The quantity is coerced to a positive finite float and the date to the canonical YYYY-MM-DD string; an existing item with the same name (case-insensitive) is updated rather than duplicated, and a new item gets the next ing-NNN id.

State of the data at each stage

Example values come from the automated end-to-end test (T12), which drives the real CLI with scripted keystrokes and a hardcoded Gemini response.

State	Type	Transformation	Example value
① Raw keystrokes	str	input() inside io_manager only	"4", "1", "20", "nuts", "", ""
② Request record	dict	prompt_recipe_generation_criteria(): typed, trimmed, comma-split	{"meal_type": "Breakfast", "max_cook_time_mins": 20, "allergies": ["nuts"], "mandatory_ingredients": [], "additional_notes": ""}
② Inventory records	list[dict]	load_inventory(): JSON → objects; non-dict rows dropped	[{"id": "ing-001", "name": "Eggs", "quantity": 6, "unit": "pcs", "location": "Fridge", "expiry_date": "2026-10-09"}, ...]
③ Prompt string	str	build_prompt(record): inventory + constraints + required JSON schema	"...- Eggs (6 pcs, Location: Fridge, Expiry: 2026-10-09) ... Restrictions to avoid: nuts ... You MUST respond strictly with a valid JSON object..."
④ Request payload	bytes	call_api(): JSON-encoded POST, key in x-goog-api-key header	{"contents": [{"parts": [{"text": "<prompt ③>"}]}, {"generationConfig": {"responseMimeType": "application/json", "temperature": 0.0}}
⑤ Response envelope	dict	json.loads() of the HTTP body	{"candidates": [{"content": {"parts": [{"text": "{\\"recipe_name\\": \\"Spinach & Cheddar Omelette\\", ...}"}]}, {"finishReason": "STOP"}]}
⑥ Validated recipe	dict	parse_response() → validate_response(): 8 fields type-checked, range-checked, trimmed	{"recipe_name": "Spinach & Cheddar Omelette", "estimated_cook_time_mins": 15, "ingredients_used": ["3 Eggs", "50g Cheddar Cheese, shredded", "1 cup fresh Spinach"], "missing_ingredients": [], "required_tools": [...], "instructions": [...], ...}
⑦ Evaluation record	dict	evaluate(record) → score(), route()	{"outcome": "ACCEPTED", "status_code": "OPTIMAL_MATCH", "match_ratio": 1.0, "expiry_priority_score": 100.0, "rescued_items": [Eggs, Cheddar Cheese, Spinach], "waste_diverted_grams": 950, ...}
⑧ History record → D2	dict → JSON	save(record): append, write .tmp, atomic os.replace()	{"recipe": {ⓐ}, "evaluation": {ⓑ}, "created_at": "2026-10-08 14:52:10"} appended to data/recipe_history.json
⑨ Recipe card	text	display_recipe_card(ⓐ, ⓑ)	Outcome Status : [ACCEPTED] - OPTIMAL_MATCH · Expiry Priority : 100.0 / 100 · Food Waste Saved: ~950 grams

How route() turns the evaluation into a verdict (⑦)

#	Condition (checked in order; first match wins)	Outcome	status_code
1	Any ingredient matches a user allergen or a synonym of one	REJECTED	ALLERGEN_CONTAMINATION
2	Ingredient match ratio < 50 % (pantry staples such as salt, black pepper and cooking oil are not counted)	REJECTED	INSUFFICIENT_INGREDIENTS
3	A must-include ingredient is absent from the recipe	FLAGGED	MISSING_MANDATORY_INGREDIENT
4	Cook time exceeds the user's limit	FLAGGED	EXCEEDS_TIME_LIMIT
5	Some ingredients missing, but ratio ≥ 50 %	FLAGGED	MISSING_SOME_INGREDIENTS
6	Otherwise	ACCEPTED	OPTIMAL_MATCH

4. Exception Handling Matrix (Detailed)

Every scenario behind the four required failure classes in Table 3. The **Test** column names the automated test (Section 5) that triggers it offline and asserts the stated behaviour. No failure is allowed to crash the menu loop.

#	Scenario	Detected by	Handling & recovery → what the user sees	Test
A. API connection failure				
1	No API key configured	get_api_key() returns "" in process()	Fails fast, before any network call. → [ERROR] "Gemini API key is not configured...", then back to the menu.	T9 T12
2	Network down, DNS failure, refused connection, or 30 s timeout	URLError / TimeoutError caught in call_api	Returned as (False, None, msg) and logged; classified <i>retry</i> : 1.5 s back-off, 2 attempts × 3 models; nothing saved. → "AI API service temporarily unavailable..." if all fail.	T9
3	Rate limited or server busy (HTTP 429 / 5xx)	HTTPError → classify_api_error() = retry	Back off, retry the same model, then move down the cascade. → Invisible if a retry succeeds.	T9
4	Invalid or revoked key (401, 403, API_KEY_INVALID)	classify_api_error() = abort	Stops after one call, because no other model can succeed with a bad key. → "AI request rejected (check your GEMINI_API_KEY)".	T9 T12
5	Model retired or request rejected (404 / 400)	classify_api_error() = next_model	Skips the second attempt and moves to the next model in the cascade. → Invisible if a fallback model succeeds.	T9
B. Malformed API response				
6	HTTP body is not JSON, or JSON is not an object	JSONDecodeError / isinstance check in call_api	Counted as a failed attempt → retry, then cascade.	T9
7	No candidates (safety block), no parts, empty text, or a candidate that is not an object	Guard clauses + AttributeError / TypeError in parse_response	Returns (False, {}, reason) and never raises; the request loop tries again.	T8
8	Prose or truncated JSON instead of a JSON object	JSONDecodeError in parse_response	Rejected; the next valid response is used (a malformed-then-valid sequence is tested).	T8 T9
9	Valid JSON wrapped in ```json fences	strip_markdown_fences()	Fences are stripped and the JSON accepted (tolerant reader). → Normal recipe card.	T8
10	Missing field, wrong type ("about 10 minutes", true), negative or NaN cook time, empty name, ingredients or steps	validate_response() (8 required fields, types, ranges)	Rejected with a field-specific reason; invalid data never reaches the logic layer. → After the final attempt: [ERROR] with the last reason; nothing saved.	T4 T8
C. Corrupt or missing data file				
11	File or data/ folder missing (first run, fresh container), or empty file	os.path.exists and an empty-content check in load_json_data	Returns [] and creates the folder; the file is created on first save. → "Your inventory is currently empty..."	T3 T10
12	Corrupt JSON (truncated or hand-edited)	JSONDecodeError in load_json_data	Logged; the file is moved to *.corrupt-<timestamp>.bak, so the next save cannot overwrite the original bytes; the app continues with [].	T3 T10
13	Wrong shape (object instead of array, non-object rows)	isinstance filters in load_inventory / load	Invalid rows are dropped and valid rows kept.	T10
14	Unreadable file (permissions, invalid UTF-8)	OSError / UnicodeDecodeError	Logged; the default [] is returned and the app keeps running.	—
15	Write fails (read-only disk, bad path, unserialisable value)	OSError / TypeError / ValueError in save_json_data	Returns False and removes the temp file; the caller reports it. → [ERROR] "Failed to save inventory to storage."	T10
16	Crash or power loss during a save	Prevented by design	Data is written to file.tmp and then swapped in with an atomic os.replace(), so the stored file is never half-written.	T10
D. Invalid user input				
17	Blank text; non-numeric or out-of-range menu, meal or storage choice	prompt_menu_choice, prompt_string, prompt_meal_type, prompt_storage_location	Re-prompts in a loop; the allergy list is trimmed and blank entries dropped. → "Invalid option. Please choose between 1 and 7."	T11
18	Bad or impossible date (12-10-2026, 2026-02-30)	strptime ValueError in prompt_date	Re-prompts with an example; the date is stored canonically as YYYY-MM-DD.	T11 T12
19	Zero, negative, non-numeric, inf or nan quantity or cook time	prompt_positive_number (ValueError, math.isfinite, > 0)	Re-prompts. → "Number must be strictly greater than 0." / "...a finite value."	T11 T12
20	Removing an unknown item; fewer than 2 items before an AI request	remove_inventory_item flag; rule in handle_generate_recipe	Nothing is saved and no API call is made. → [WARNING] "...was not found" / "...add more items".	T12
21	Ctrl+C or closed input stream (EOF, e.g. docker run without -it)	KeyboardInterrupt / EOFError in main()	Clean exit with code 0; every action had already been saved. → "Input stream closed. Exiting ... safely."	T12

5. Automated Testing

tests/test_pipeline.py is a procedural test runner: it defines no classes and never contacts the live API. It uses **hardcoded sample Gemini responses** (tests/sample_ai_responses.py), each a full generateContent envelope as the real API returns it, and substitutes call_api and the keyboard reader with scripted stand-ins. It exits non-zero on any failure, so CI and Docker can gate on it.

ID	Test	What it proves
T1	Zero-classes constraint	A regex scan of every .py file finds no class definition
T2	print / input localisation	A grep-strict scan finds print(and input(only in src/io_manager.py
T3	Data persistence & resilience	save/load round trip, query(filter_fn) incl. a raising filter, expiry query, corrupt-file fallback
T4	AI schema validation	A valid payload is accepted; a missing field, wrong type or empty list is rejected
T5	Logic multi-condition rules	Allergen gate, score(), and the ACCEPTED / FLAGGED / REJECTED branches of evaluate()
T6	Logic rules on sample AI responses	Six hardcoded responses → parse_response → validate_response → evaluate reach every verdict, with exact score (100), ratio (0.6) and waste (950 g)
T7	Logic helper functions	route() rule order, fixed-date expiry maths, fuzzy matching, staples, unit conversion
T8	Malformed AI responses	Nine malformed envelopes are rejected without raising; a fenced-JSON response is accepted
T9	Offline API failure handling	Network down, timeout, non-JSON body, invalid key, 404 cascade, 429 recovery, missing key, real refused socket
T10	Corrupt & unwritable files	Quarantine to .bak, empty and wrong-shape files, atomic save, write failure returns False
T11	I/O re-prompting	Every prompt rejects blank, text, out-of-range, bad-date and inf/nan input until a valid value is given
T12	End-to-end pipeline (offline)	Drives the real CLI: add item → JSON file; recipe → history record; history query; API 401 → nothing saved; EOF → clean exit

Result: 12/12 pass locally (Python 3.14, Windows 11) and inside the Docker image (Python 3.12-slim, Linux), with **no API key and no network**: docker compose run --rm test.

6. DevOps: Version Control, CI and Docker

Git workflow

- Work happens on feature branches (e.g. feature/part1-deliverables) and is merged into main by pull request once CI passes.
- One logical change per commit, with a descriptive message, e.g. *"quarantine corrupt data files and make JSON saves atomic"*.
- .env is git-ignored (only .env.example is committed), and the API key is sent in a header, never in a URL.

Docker

- Dockerfile sits in the repository root and builds on python:3.12-slim with zero third-party packages (standard library only), giving a 43 MB compressed image.
- /app/data is a volume, so data persists between runs. docker compose run --rm app starts the app; the test service needs no .env.

CI pipeline (.github/workflows/ci.yml)

GitHub Actions runs on every push and on every pull request into main:

1. **Lint & test** on Python 3.10 and 3.12: ruff (syntax errors, undefined names), byte-compile, then the full offline suite with an empty API key.
2. **Docker**, after step 1 passes: build the image, run the suite *inside* the container, and smoke-test that the CLI starts and exits cleanly.

A red job blocks the merge, so main always builds, passes its tests and runs in a container.

7. Known Limitations

- Ingredient matching is lexical (substring and plural stemming), so "coconut milk" matches "milk". Semantic or AI-assisted matching would be more precise.
- The waste figure is a heuristic: about 100 g per countable unit, with the whole stock of each rescued item counted. Quantities are not decremented after cooking.
- Pantry staples do not lower the match ratio, but they are still listed as missing and can therefore produce an informational FLAGGED verdict.